



Lecture 06

Interfaces and Abstraction

Basics of Internet Technologies 2

One contract, several possible implementations

Learning Objectives

- Explain abstraction in practical terms.
- Define and implement a C# interface.
- Use an interface variable to access different implementations.
- Explain polymorphism with a common contract.
- Use simple composition through a constructor.
- Refactor the expense tracker to depend on `IExpenseRepository`.
- Compare memory and JSON storage without rewriting menu code.
- Identify when an interface is useful and when it is unnecessary.

Connection to Lecture 05

- Lecture 05 gave an expense a real type: Expense.
- The console program still did too many different jobs.
- It managed menu input, application rules, and storage details.
- Today we create a small boundary around storage.
- We do not build full layers yet; first we learn the reason for the boundary.

Three-Hour Lesson Plan

- 20 min - the storage-change problem and abstraction.
- 25 min - interface syntax and implementing a contract.
- 25 min - polymorphism with notifier and shape examples.
- 30 min - IExpenseRepository with memory storage.
- 30 min - JSON implementation of the same contract.
- 25 min - composition in ExpenseTrackerApp.
- 25 min - live refactoring, exercises, and database connection.

The Storage Problem

```
List<Expense> expenses = LoadExpenses(path);  
  
// Program.cs reads, changes, and saves data itself.  
File.WriteAllText(path, json);
```

- The menu code knows a specific storage detail: a JSON file path.
- If we later want a database, Program.cs must change in many places.
- We want the application to ask for storage actions, not file operations.

Abstraction

- Abstraction means focusing on what code needs, not every detail of how it happens.
- The expense application needs to get expenses, add an expense, and delete an expense.
- It does not need to know whether those actions use a list, JSON, or SQL.
- An interface can describe this small promise.

What Is an Interface?

```
public interface INotifier
{
    void Send(string message);
}
```

- An interface is a contract.
- It lists members that an implementing class must provide.
- The interface usually describes a capability: notify, store, calculate, send.
- It does not contain the concrete implementation in this beginner example.

Naming Convention

- C# interfaces commonly begin with a capital I.
- Examples: INotifier, IShape, IExpenseRepository.
- The I makes a contract easy to recognize in code.
- The rest of the name should describe what the contract does.

Implementing an Interface

```
public class ConsoleNotifier : INotifier
{
    public void Send(string message)
    {
        Console.WriteLine("Console: " + message);
    }
}
```

- A colon means ConsoleNotifier implements INotifier.
- The class must provide every required interface member.
- The method signature must match the contract.
- The body is the implementation detail.

A Second Implementation

```
public class UppercaseNotifier : INotifier
{
    public void Send(string message)
    {
        Console.WriteLine(message.ToUpperInvariant());
    }
}
```

- The interface stays the same.
- The implementation can behave differently.
- Both classes can be used where INotifier is expected.

Interface Variable

```
INotifier notifier = new ConsoleNotifier();  
notifier.Send("Welcome");  
  
notifier = new UppercaseNotifier();  
notifier.Send("Welcome");
```

- The variable type is the contract, not a concrete class.
- The object behind it can change.
- Only INotifier members are visible through this variable.
- This is a deliberate limit: callers use the promised capability.

Polymorphism

- Polymorphism means one interface type can represent different concrete objects.
- The calling code sends the same message: `notifier.Send(...)`.
- Each object decides what that action actually does.
- The call site does not need a long if or switch for every implementation.

Polymorphism With a List

```
List<IShape> shapes = new List<IShape>
{
    new Rectangle(4, 3),
    new Circle(2)
};

foreach (IShape shape in shapes)
    Console.WriteLine(shape.GetArea());
```

- A list can hold values with a shared interface.
- Rectangle and Circle have different area formulas.
- The loop calls one common method on every item.

Interface Property

```
public interface IShape
{
    string Name { get; }
    double GetArea();
}
```

- Interfaces can require properties as well as methods.
- get-only means callers can read Name but cannot assign it through IShape.
- The class chooses how to provide the property.

Concrete Class: Rectangle

```
public class Rectangle : IShape
{
    public string Name => "Rectangle";
    public double Width { get; }
    public double Height { get; }

    public double GetArea()
    {
        return Width * Height;
    }
}
```

- Rectangle has its own state: Width and Height.
- It fulfils the shared IShape contract.
- Its details do not leak into code that only needs IShape.

Concrete Class: Circle

```
public class Circle : IShape
{
    public string Name => "Circle";
    public double Radius { get; }

    public double GetArea()
    {
        return Math.PI * Radius * Radius;
    }
}
```

- Circle has different data and a different formula.
- No change is needed in the foreach loop.
- This is a useful sign that the contract captures the right common behavior.

Interface Is Not a Class

Interface

- Defines required members
- Describes a capability
- Cannot be created with new
- Can have many implementations

Class

- Contains data and behavior
- Creates objects with new
- Can implement interfaces
- Can have constructors

Do Not Create Interfaces Automatically

- An interface is useful when there are real alternative implementations or a real boundary.
- Two classes with identical behavior do not need an interface just because they exist.
- Begin with a concrete class when the problem is simple.
- Introduce a contract when it removes a real dependency or enables a useful variation.

The Expense Storage Contract

```
public interface IExpenseRepository
{
    List<Expense> GetAll();
    void Add(Expense expense);
    bool DeleteById(int id);
}
```

- Repository is a name for code that manages access to stored data.
- The contract states what the application needs from storage.
- It does not say whether the data lives in a list, JSON file, or database.

Read the Contract Carefully

GetAll

- Returns current expenses
- Caller can list, filter, and summarize

Add

- Stores one new Expense
- No file or database detail exposed

DeleteById

- Returns true when found
- Returns false when no matching id exists

In-Memory Implementation

```
public class InMemoryExpenseRepository : IExpenseRepository
{
    private readonly List<Expense> expenses = new();

    public List<Expense> GetAll() => expenses;
    public void Add(Expense expense) => expenses.Add(expense);
    // DeleteById removes a matching item.
}
```

- This version is fast and simple for a demo.
- Data disappears when the program stops.
- It is useful for learning and for tests later.

JSON Implementation

```
public class JsonExpenseRepository : IExpenseRepository
{
    private readonly string filePath;
    private readonly List<Expense> expenses;

    public JsonExpenseRepository(string path)
    {
        filePath = path;
        expenses = Load();
    }
}
```

- This version owns the file path and loading logic.
- The rest of the application does not see `File.ReadAllText`.
- The constructor prepares the repository for use.

Saving Inside the Repository

```
public void Add(Expense expense)
{
    expenses.Add(expense);
    Save();
}
```

- The repository decides when its storage must be updated.
- The app requests Add; it does not request JSON serialization.
- This keeps persistence details in one location.

Choosing an Implementation

```
interface IRepository {  
    void addExpense(Expense expense);  
    List<Expense> getExpenses();  
}  
  
class InMemoryExpenseRepository implements IRepository {  
    List<Expense> expenses = new ArrayList<>();  
    void addExpense(Expense expense) {  
        expenses.add(expense);  
    }  
    List<Expense> getExpenses() {  
        return expenses;  
    }  
}  
  
class JsonExpenseRepository implements IRepository {  
    String dataPath;  
    void addExpense(Expense expense) {  
        // ...  
    }  
    List<Expense> getExpenses() {  
        // ...  
    }  
}  
  
ExpenseTrackerApp app = new ExpenseTrackerApp(repository);
```

- Only the composition point chooses the concrete storage type.
- The rest of the app receives the shared interface.
- Later configuration and dependency injection will improve this setup.

Composition

- Composition means one object uses another object as part of its work.
- ExpenseTrackerApp has an IExpenseRepository.
- It does not inherit from a repository. It uses one.
- This often models real software more clearly than a deep inheritance tree.

Constructor Receives a Dependency

```
public class ExpenseTrackerApp
{
    private readonly IExpenseRepository repository;

    public ExpenseTrackerApp(IExpenseRepository repository)
    {
        this.repository = repository;
    }
}
```

- The app receives the object it needs through its constructor.
- readonly prevents replacing the reference after construction.
- The word dependency simply means something this class needs to do its job.

Using the Contract in the App

```
List<Expense> expenses = repository.GetAll();  
int nextId = expenses.Count == 0  
    ? 1  
    : expenses.Max(expense => expense.Id) + 1;  
  
repository.Add(newExpense);
```

- The app calls contract methods only.
- It never calls `File.WriteAllText` or `JsonSerializer`.
- Both repository versions support this exact code.

Before and After

Before Lecture 06

- Program.cs loads JSON
- Program.cs saves JSON
- Menu is tied to files
- Database requires wider change

After Lecture 06

- App uses IExpenseRepository
- Repository owns storage details
- Menu works with either implementation
- Future SQL version fits the contract

What Changes for SQL Later?

- The interface can stay nearly the same.
- A future `SqlExpenseRepository` can implement `IExpenseRepository`.
- `GetAll` can run `SELECT`; `Add` can run `INSERT`; `DeleteById` can run `DELETE`.
- The console application can keep using the same repository methods.
- We will learn SQL and database access before writing that implementation.

Common Beginner Errors

- Writing new `INotifier()` - interfaces cannot be created directly.
- Forgetting to implement one required interface member.
- Adding methods to a class but not to the interface when callers need them.
- Using a concrete type everywhere after creating an interface.
- Creating an interface for every class without a real need.
- Confusing composition with inheritance.

Guided Exercise A

- Create an IMessageSender interface with Send(string text).
- Implement ConsoleMessageSender and BracketMessageSender.
- Write one method that accepts IMessageSender.
- Call the same method with both implementations.
- Explain which code knows the concrete class and which code knows only the contract.

Guided Exercise B

- Create an IDiscount interface with GetDiscount(decimal price).
- Implement NoDiscount and PercentageDiscount.
- Store both implementations in a List<IDiscount>.
- Calculate the result for the same price with each implementation.
- Do not use a large switch statement for discount types.

Student Project Increment

- Keep your entity class from Lecture 05.
- Create an interface that describes only storage actions your project actually needs.
- Implement an in-memory repository first.
- Move your existing JSON load/save code into a JSON repository.
- Make the menu or app class depend on the interface, not the JSON class.

Shared Database-Course Increment

- Write storage actions in plain English: read all, add, delete by id.
- For each action, identify its future SQL statement: SELECT, INSERT, DELETE.
- Use the same table and entity designed in the database lesson.
- Do not connect C# to SQL yet; the goal is to see the future boundary clearly.

Exit Ticket

- What problem does IExpenseRepository solve?
- Why can IExpenseRepository refer to either memory or JSON storage?
- What is polymorphism in the notifier example?
- Why is ExpenseTrackerApp composed with a repository?
- When would adding an interface be unnecessary?



Next Time

Strategy Pattern and Dependency Thinking

We will choose behavior deliberately at runtime.

The interface becomes a practical tool rather than just syntax.